Performance Trade-offs in Static and Dynamic Thread Pool Architectures

Harshil Patel
University of California, Davis
Department of Computer Science
USA

Anugya Sharma
University of California, Davis
Department of Computer Science
USA

Kaichi Xie
University of California, Davis
Department of Computer Science
USA

Alfredo Ortiz
University of California, Davis
Department of Computer Science
USA

1 Introduction & Motivation

The advancements in multicore architecture and multiprocessor computers are increasing and ongoing [1]. Modern concurrent systems have to choose between static thread pools (pre-created threads) and dynamic thread pools (threads created on demand), but they lack the empirical data to make informed decisions. Static thread pools waste memory during low load and idle periods which results in a constant overhead regardless of the workload. Dynamic thread pools avoid this waste by creating threads only when needed, but they incur overhead for thread creation and destruction, which can lead to performance degradation under high load. In this project, we want to evaluate the performance trade-offs between static and dynamic thread pools under varying workloads and system configurations and answer the question: At what point does the overhead of dynamic thread creation outweigh its memory savings compared to static thread pools?

This tradeoff can directly impact the performance and costs of cloud-based systems [3]. A web service handling variable traffic has to decide between a static pool with constant resource usage or a dynamic pool that adapts to demand but may suffer from latency spikes during high traffic. As cloud providers charge based on resource usage, understanding this tradeoff can lead to significant cost savings while maintaining performance. Apart from costs, this question also affects other aspects of system design such as database connection pools, task scheduling in distributed systems, etc which all use thread pools to manage concurrency. Yet, there is a lack of comprehensive studies that quantify the performance implications of static vs dynamic thread pools across different workloads and system configurations. By systematically evaluating these trade-offs, we can provide practical guidelines for system architects to choose the right thread pool strategy based on their specific workload and performance requirements.

2 Background & Related Work

Existing research mainly focuses on optimizing thread pool sizes rather than comparing different thread pool architectures. Current heuristics that are used to determine the number of threads are flawed [7]. In our preliminary research, we find that there have been few researches on dynamic model for thread size. One of the first such model was the Watermark model which adjusts the number of threads according to the current number of incoming requests [5]. The issue with this model is that while it solves the problem of "efficient usage of resources", it doesn't efficiently obtain the required number of threads in advance [4]. Furthermore, there are also prediction based models to determine thread pool size, but in such models, only factors such as the request rate or number of worker threads is considered in the management [6]. One of the research that has proposed dynamic thread pool model(prediction based), themselves point out that the model lacks testing for a longer period of time and on general server environment which provides various services simultaneously [7]. Lee et al. [6] adds to the prediction model by proposing what they call the "Trendy exponential moving average model" and again the issue with this model seems to be that it doesn't have the desired accuracy in its prediction.

A comprehensive study comparing the tradeoffs between static and dynamic thread pools under varying workloads and system configurations is missing. This tradeoff can directly impact the performance and costs of cloud-based systems [3]. Freire et al. [3] compare global thread pools (single pool and queue for all tasks) against local thread pools (each task has its own dedicated pool and queue). Their study simulates these configurations under high workloads to demonstrate that optimized local pools result in a lower average makespan when compared to global pools. While Freire et al. [3] provide insights into thread pool configurations, they focus on optimal distribution of a fixed number of threads, while our work aims to compare static vs dynamic thread pool architectures across varying workloads and system settings. By systematically evaluating these trade-offs, we can provide practical guidelines for system architects to choose the right thread pool strategy based on their specific workload and performance requirements.

3 Challenges

When implementing two different thread pools to compare their performance, one challenge is to make sure that the comparison is fair. We need to make sure that both implementations are the same in terms of how they execute instructions and the only difference is how they manage the threads. As shown by Desrochers [2], that the throughput gap between a lock-free queue and lock_based queues can range from 14x to 200x depending on the platform. We need to make sure that both implementations use the same data structures such as queues, locks, etc and the same algorithms to manage to schedule the tasks and threads.

4 Proposed Solution

We address the shortcomings mentioned in 2 by implementing minimal static and dynamic thread pool architectures in C++ that differ only in their thread lifecycle management policies, while sharing the same task scheduling and execution logic. The static thread pool will pre-create a fixed number of threads at initialization, which will remain active throughout the lifetime of the pool. The dynamic thread pool will start with zero threads and will create new threads on-demand when tasks arrive. Idle threads will be terminated after a configurable timeout period to free up resources. Importantly, both implementations will use the same task queue, task scheduling algorithm (e.g., FIFO), and other shared components to ensure a fair comparison. This approach will allow us to isolate architectural differences from quality unlike comparing off the shelf implementations like Intel TBB or Boost, which may have optimizations that effect performance independently of thread pool policy.

4.1 Evaluation

To evaluate our proposed thread pool implementations, we will benchmark them under three traffic patterns, each defined by a task arrival process that reflects different traffic scenarios:

- Steady load: a constant arrival rate of X tasks/second for the full duration of the run.
- Burst Load: a burst of X tasks/second followed by a Y-second idle period, repeated in cycles.
- Ramping Load: alternating phases on X tasks/second and Y tasks/second in repeated cycles.

We will also vary the number rate of incoming tasks to simulate different levels of system load. To isolate how task characteristics interact with thread-pool policy, we will apply two controlled micro-benchmarks:

- Memory-based tasks: $N \times N$ matrix multiplications, where N is tunable to scale memory pressure.
- Latency-based tasks: a synthetic task that blocks for a configurable sleep duration T to emulate downstream I/O or service latency.

For each task type, we evaluate four benchmark intensities: Light, Medium, Heavy, and Mixed. For latency-based tasks, each request performs a blocking sleep with $T \in \{100\,\mu s, 1\,ms, 10\,ms\}$ to represent Light, Medium, and Heavy service times, respectively. For memory-based tasks, we use $N \times N$ matrix multiplication with $N \in \{128, 256, 512\}$ to represent Light, Medium, and Heavy working-set sizes, respectively. The Mixed workload captures heterogeneity by combining light, medium, and heavy tasks in varied proportions (e.g., 50% light, 30% medium, 20% heavy).

We will evaluate the following experiment matrix (Table 1):

Dimension	Values
Traffic Patterns	Steady load, Burst Load, Ramping Load
Task Types	Memory-based, Latency-based
Task Intensities	Light, Medium, Heavy, Mixed
Pool Types	Static, Dynamic
Core Counts	4 cores, 16 cores

Table 1: Experiment configuration matrix.

For each task arrival pattern, we will measure throughput (tasks completed per second), latency (p50, p95, p99 time from task submission to completion), PSS Memory (Proportional Set Size attributable RAM used by the process), and thread count (active threads over time). We will fix the CPU resources available to the benchmark by restricting it to exactly 4 or 16 cores using Linux CPU affinity (e.g., tasket). For each configuration, we run the benchmark for 120 seconds and repeat the process three times. we will then aggregate the metrics across the three trials and report the mean and variance.

5 Expected Results

For out project, we expect to see that the dynamic thread pool with outperform the static thread pool in scenarios with variable workloads, such as bursty or ramping traffic patterns. This is because the dynamic pool can adapt to changing demand by creating or terminating threads as needed, leading to better resource utilization. In contrast, the static thread pool might perform well under steady load conditions where the fixed number of threads can handle the incoming tasks without significant queuing delays. We also expect the dynamic thread pool to incur some overhead due to thread creation and destruction, which may lead to higher latency in scenarios with frequent load changes. However, we expect that this overhead will be offset by the improved responsiveness and reduced queuing delays in high load and variable traffic scenarios. In terms of memory usage, we expect the dynamic thread pools to use less overall memory when compared to the static thread pools during low load periods, as idle threads in the dynamic thread pool can be terminated to free up resources.

6 Timeline

We have broken down our timeline into weeks with six weeks to finish and present the project (Table 2).

Week	Tasks
Week 1 (2026-2-3)	Research dynamic and static thread pools, experiment with code to understand implementation
Week 2 (2026-2-8)	Start implementing the thread pools, traffic generation scripts and evaluation scripts, continue research as needed
Week 3 (2026-2-15)	Continue working on code, verify progress aligns with plan, fix bugs
Week 4 (2026-2-22)	Complete implementation and start evaluation
Week 5 (2026-3-1)	Complete results, write research paper, create presentation slides and practice
Week 6 (2026-3-8)	Finalize results, paper, and slides. Submit project and present

Table 2: Project timeline.

References

[1] George K Adam. 2022. Co-design of multicore hardware and multithreaded software for thread performance assessment on an FPGA. *Computers* 11, 5 (2022), 76.

[2] Cameron Desrochers. 2014. A Fast General Purpose Lock-Free Queue for C++. https://moodycamel.com/blog/2014/a-fast-general-purpose-lock-free-queue-for-c++ Accessed: 2026-02-02.

[3] Daniela L Freire, Angela Mazzonetto, Rafael Z Frantz, Fabricia Roos-Frantz, Sandro Sawicki, and Vitor Basto-Fernandes. 2021. Performance evaluation of thread pool

configurations in the run-time systems of integration platforms. *International Journal of Business Process Integration and Management* 10, 3-4 (2021), 318–329.

[4] DongHyun Kang, Saeyoung Han, SeoHee Yoo, and Sungyong Park. 2008. Prediction-based dynamic thread pool scheme for efficient resource usage. In *2008 IEEE 8th International Conference on Computer and Information Technology Workshops*. IEEE, 159–164.

[5] Ji Hoon Kim, Seungwok Han, Hyun Ko, and Hee Yong Youn. 2007. Prediction-based dynamic thread pool management of agent platform for ubiquitous computing. In *International Conference on Ubiquitous Intelligence and Computing*. Springer, 1098–1107.

[6] Kang-Lyul Lee, Hong Nhat Pham, Hee-seong Kim, Hee Yong Youn, and Ohyoung Song. 2011. A Novel Predictive and Self – Adaptive Dynamic Thread Pool Management. In *2011 IEEE Ninth International Symposium on Parallel and Distributed Processing with Applications*. 93–98. doi:10.1109/ISPA.2011.61

[7] Yibei Ling, Tracy Mullen, and Xiaola Lin. 2000. Analysis of optimal thread pool size. *ACM SIGOPS Operating Systems Review* 34, 2 (2000), 42–55.